CSE 320 - Computer Networks C

LAB Session 4

13.04.2026

Simple TCP Communication Lab in C

TCP Login Check + Role Detector

Goal: By the end of the lab, students should be able to:

1. create a TCP socket in C
2. bind and listen on a server
3. connect from a client
4. send and receive data over TCP
5. run both programs from the Windows command line

An example TCP system:

1. **server** waits for a connection
2. **client** connects and sends a message
3. **server** prints the message and replies
4. **client** prints the reply

Server:

- listens on `127.0.0.1:8083`
- accepts one client
- receives a username and password in one message
- checks whether credentials are valid
- sends one of these responses:
    - **Login successful | Role: admin**
    - **Login successful | Role: user**
    - **Login failed**

Valid credentials

Use these fixed accounts:

- `admin 1234`
- `student 2024`

Client:

- connects to the server
- asks user for username
- asks user for password
- combines them into one message
- sends to server
- prints server response

1. Prepare files

- **`server.c, client.c`**

2. Add required headers

server.c

```c
#include <stdio.h>
#include <string.h>
#include <winsock2.h>

#pragma comment(lib, "ws2_32.lib")

```

client.c

```c
#include <stdio.h>
#include <string.h>
#include <winsock2.h>

#pragma comment(lib, "ws2_32.lib")

```

You'll need c socket functions and window's own socket library.

3. Start WinSock

Both programs will require the following

```
WSADATA wsa;
WSAStartup(MAKEWORD(2,2), &wsa);
```

4. Create a TCP Socket

AF_INET IPv4, SOCK_STREAM TCP

```
SOCKET s = socket(AF_INET, SOCK_STREAM, 0);
```

5. Start main(

Create the main functions in both programs (client.c and server.c)

```
5   int main()
6   {
7
8   }
9
```

A. Server Side

1. Declare Variables

```
8       WSADATA wsa;
9       SOCKET server_socket, client_socket;
10      struct sockaddr_in server, client;
11      int client_size;
12      char buffer[1024] = {0};
13      char username[100], password[100];
14      char reply[200];
15
```

2. Initialize Winsock

```
15
16      if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) {
17          printf("WSAStartup failed.\n");
18          return 1;
19      }
20
```

3. Create Server Socket and Set Server Address

In the server program

```
21      server_socket = socket(AF_INET, SOCK_STREAM, 0);
22      if (server_socket == INVALID_SOCKET) {
23          printf("Could not create socket.\n");
24          WSACleanup();
25          return 1;
26      }
27
28      server.sin_family = AF_INET;
29      server.sin_addr.s_addr = inet_addr("127.0.0.1");
30      server.sin_port = htons(8083);
31
```

4. Server “Bind”

Bind operation that’s specific to the server model

```
32      if (bind(server_socket, (struct sockaddr *)&server, sizeof(server)) == SOCKET_ERROR) {
33          printf("Bind failed.\n");
34          closesocket(server_socket);
35          WSACleanup();
36          return 1;
37      }
```

5. “Listen”

Operations to listen from server side

```
39      listen(server_socket, 1);
40      printf("Server listening on 127.0.0.1:8083...\n");
41
```

6. When client is sending a message, server “Accept”s it

```
42      client_size = sizeof(struct sockaddr_in);
43      client_socket = accept(server_socket, (struct sockaddr *)&client, &client_size);
44      if (client_socket == INVALID_SOCKET) {
45          printf("Accept failed.\n");
46          closesocket(server_socket);
47          WSACleanup();
48          return 1;
49      }
```

7. “Receive” message from the Client

```
51      recv(client_socket, buffer, sizeof(buffer), 0);
52      printf("Received: %s\n", buffer);
```

8. Parse Input and Reply Client

```
54      sscanf(buffer, "%s %s", username, password);
55
56      if (strcmp(username, "admin") == 0
57                  && strcmp(password, "1234") == 0) {
58          strcpy(reply, "Login successful | Role: admin");
59      } else if (strcmp(username, "student") == 0
60                  && strcmp(password, "2024") == 0) {
61          strcpy(reply, "Login successful | Role: user");
62      } else {
63          strcpy(reply, "Login failed");
64      }
65
66      send(client_socket, reply, strlen(reply), 0);
```

9. Close Existing Sockets for both Server listen and Message

```
54      closesocket(client_socket);
55      closesocket(server_socket);
56      WSACleanup();
57
58      return 0;
```

B. Client Side

1. Declare Variables

```
    WSADATA wsa;
    SOCKET s;
    struct sockaddr_in server;
    char username[100];
    char password[100];
    char message[220];
    char server_reply[200] = {0};
```

2. Initialize Winsock

```
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) {
        printf("WSAStartup failed.\n");
        return 1;
    }
```

3. Create Socket and Set Server Info

```
    s = socket(AF_INET, SOCK_STREAM, 0);
    if (s == INVALID_SOCKET) {
        printf("Could not create socket.\n");
        WSACleanup();
        return 1;
    }

    server.sin_family = AF_INET;
    server.sin_addr.s_addr = inet_addr("127.0.0.1");
    server.sin_port = htons(8083);
```

4. Client “Connect”s

```
    if (connect(s, (struct sockaddr *)&server, sizeof(server)) < 0) {
        printf("Connection failed.\n");
        closesocket(s);
        WSACleanup();
        return 1;
    }
```

5. Get message from user and Remove newline from `fgets`

```
38
39      printf("Enter username: ");
40      scanf("%99s", username);
41
42      printf("Enter password: ");
43      scanf("%99s", password);
44
45      sprintf(message, "%s %s", username, password);
```

6. **Send Message**

```
47      send(s, message, strlen(message), 0);
```

7. **Receive reply and Print**

```
48      recv(s, server_reply, sizeof(server_reply), 0);
49
50      printf("Server replied: %s\n", server_reply);
```

8. **Close Socket and terminate**

```
46      closesocket(s);
47      WSACleanup();
48
49      return 0;
```

C. Compile and Run

Visual Studio (on **Developer Command Prompt for Visual Studio**)
cl server.c ws2_32.lib
cl client.c ws2_32.lib

or

Use MinGW gcc compile
https://www.mingw-w64.org/downloads/#msys2
https://www.msys2.org/

In the program file, run the mingw64.exe and type “pacman -S mingw-w64-x86_64-gcc” and type Y for installation

Locate to the code files. E.g. cd /c/Users/manol/lab4 (instead of C:\, /c/..)

gcc server.c -o server -lws2_32
gcc client.c -o client -lws2_32

Open Two terminal windows and run each program with their names
Terminal 1 -> server
Terminal 2 -> client

For MinGW